Soluciones: Estructuras y paso por referencia

Programación I, Semana 9

Ejercicios obligatorios

Ejercicio 1: El goleador

```
int posicionGoleador(Jugador e[], int n) {
    int mejor = 0;
    for (int i = 1; i < n; i++) {
        if (e[i].goles > e[mejor].goles) {
            mejor = i;
        }
    }
    return mejor;
}
```

Código de `main`:

```
int g = posicionGoleador(equipo, N);
std::cout << equipo[g].nombre;          // imprime Sara
```

También es correcto en una línea: `std::cout << equipo[posicionGoleador(equipo, N)].nombre;`.
Por qué: se compara el campo `goles` del jugador `i` con el del mejor hasta ahora. Al guardar la posición se tiene acceso a todo el registro: nombre, goles y partidos.
Errores frecuentes:

- `mejor = e[i].goles;`: guarda los goles, no la posición. Después no se puede saber el nombre.
- `e[i].goles > mejor`: compara goles con una posición.
- `equipo.nombre[g]`: el índice va antes del punto, `equipo[g].nombre`.

Ejercicio 2: Seguimiento

`i`	`e[i].nombre`	`¿== "Juan"?`	`pos`
inicio			-1
0	Leo	no	-1
1	Ana	no	-1
2	Juan	sí	2

La función devuelve **2**. Al pasar a `i = 3`, `pos` ya no es -1 y el ciclo termina sin revisar a Sara.
Con `"Pepe"` revisa los 4 jugadores, no lo encuentra y devuelve **-1**.

Ejercicio 3: ¿Qué imprime?

a) Imprime `3 10`. `Jugador b = a;` copia todos los campos. Desde ese momento son dos jugadores independientes: cambiar `b.goles` no cambia `a.goles`.
b) Imprime `9`.

- `sumarGol(p)` recibe una copia: le suma el gol a la copia y `p` sigue con 7.
- Cada `sumarGolRef(p)` recibe el original: 7 pasa a 8 y luego a 9.

c) Imprime `1 2`. `a` es una copia de `x`, así que `x` no cambia. `b` es otro nombre para `y`, así que `y` pasa a 2.

Ejercicio 4: Encontrar los errores

N.º	Error	Corrección
1	Falta el punto y coma después de la llave que cierra el `struct`.	`};`
2	`sumarGol(Jugador j)` recibe una copia: el gol no llega al arreglo.	`void sumarGol(Jugador& j)`
3	`equipo[i].nombre = "Leo"` asigna en vez de comparar.	`equipo[i].nombre == "Leo"`
4	`equipo.goles[0]`: el índice va antes del punto.	`equipo[0].goles`

Ya corregido, Leo pasa de 7 a 8 goles y el programa imprime **8**.

Ejercicios intermedios

Ejercicio 5: Registrar un partido

```
void registrarPartido(Jugador& j, int golesAnotados) {
    j.partidos = j.partidos + 1;
    j.goles = j.goles + golesAnotados;
}

double promedioGoles(Jugador j) {
    if (j.partidos == 0) {
        return 0;
    }
    return static_cast<double>(j.goles) / j.partidos;
}
```

Juan empieza con 2 goles en 7 partidos. Después de `registrarPartido(equipo[2], 2)` tiene 4 goles en 8 partidos, y el promedio es 4 / 8 = **0.5**.

¿Por qué uno lleva `&` y el otro no? `registrarPartido` debe **modificar** al jugador del arreglo: sin `&` cambiaría una copia. `promedioGoles` solo **lee** los campos y devuelve un resultado, así que una copia basta.

Errores frecuentes:

- `return j.goles / j.partidos;`: división entera. 4 / 8 daría 0.
- Olvidar el caso de 0 partidos: dividir entre 0.

Reto

Ejercicio 6: El mejor promedio

```
Jugador mejorPromedio(Jugador e[], int n) {
    int mejor = 0;
    for (int i = 1; i < n; i++) {
        if (promedioGoles(e[i]) > promedioGoles(e[mejor])) {
            mejor = i;
        }
    }
    return e[mejor];
}
```

Promedios del equipo original: Leo 0.70, Ana 1.33, Juan 0.29, Sara 0.75. Devuelve a **Ana**.
¿Por qué no es Sara? Sara tiene más goles (9), pero en 12 partidos. Ana tiene 8 goles en solo 6 partidos: anota más por partido. «Más goles» y «mejor promedio» son preguntas distintas.
Detalle: una función puede devolver un `struct` completo, igual que devuelve un `int`. Quien la llama puede guardarlo: `Jugador m = mejorPromedio(equipo, N);` y luego usar `m.nombre`.